

AWS Cloud Project: Fitness Tracker Deployment

This document describes the end-to-end implementation of a Fitness Tracker application deployed on AWS EC2 with MongoDB, AWS Secrets Manager, AWS KMS, and Amazon S3 integration.

1. Project Overview

The Fitness Tracker application is a monolithic Node.js backend deployed on an EC2 instance. It uses MongoDB for database storage and AWS services for security, secrets management, encryption, and log storage.

Fitness Tracker Application – Business Requirements

1. 📌 Business Objective

The Fitness Tracker application is designed to help users track, manage, and improve their fitness journey through a simple digital platform. It aims to provide personalized fitness tracking, goal setting, and progress monitoring in a secure and scalable cloud-based system.

2. 🎯 Business Requirements

2.1 User Management

- Users should be able to register and log in securely.
 - Role-based access (Client / Admin / Trainer).
 - Passwords must be securely encrypted.
-

2.2 Fitness Goal Tracking

- Users can set fitness goals such as:
 - Weight loss
 - Muscle gain
 - Daily activity targets
- System should track progress against goals.

2.3 Activity Management

- Users can log daily activities:
 - Workouts
 - Calories burned
 - Exercise duration
- Historical activity data should be stored and retrievable.

2.4 Progress Monitoring

- Dashboard should show:
 - Weekly / monthly progress
 - Graphical representation of fitness data
- Enable comparison between past and current performance.

2.5 Secure Data Storage

- All sensitive data must be encrypted using **AWS KMS**.
- User credentials and secrets must be stored in **AWS Secrets Manager**.
- Media/files (if any) should be stored in **Amazon S3**.

2.6 Scalability & Availability

- Application must run on cloud infrastructure (AWS EC2).
- Should support future scaling using Load Balancer and Auto Scaling Group.
- High availability with minimal downtime.

2.7 Performance Requirement

- API response time should be under 2 seconds.

- System should support concurrent users without failure.
-

3. Security Requirements

- HTTPS communication via Nginx reverse proxy.
 - IAM-based access control for AWS services.
 - Secure secret management (no hardcoded credentials).
 - Logging and monitoring using CloudWatch.
-

4. Business Alignment with Cloud Architecture

4.1 Cloud-Native Architecture

The application is deployed using AWS cloud services to ensure reliability and scalability:

- **EC2** → Hosts Node.js backend application
 - **Nginx** → Acts as reverse proxy and load handler
 - **PM2** → Ensures application uptime and process management
 - **MongoDB** → Stores user and fitness data
 - **S3** → Stores static files and media content
 - **KMS** → Encrypts sensitive data
 - **Secrets Manager** → Securely manages DB credentials
-

4.2 Business Value Mapping

Business Need	Technical Solution
Secure user authentication	Encrypted passwords + Secrets Manager
Data reliability	MongoDB + EC2 persistence
Scalability	EC2 + Load Balancer ready architecture
Performance	Nginx reverse proxy + PM2 clustering

Business Need	Technical Solution
Data security	AWS KMS encryption
File storage	Amazon S3 integration
Monitoring	CloudWatch logs

4.3 Business Impact

- Improves user fitness awareness and consistency
- Provides centralized health tracking system
- Reduces manual tracking effort
- Enables cloud-based secure access anytime, anywhere
- Supports future expansion into mobile apps and AI fitness recommendations

2. EC2 Deployment

The application is deployed on an AWS EC2 Ubuntu instance. Node.js and PM2 are used to run and manage the backend service.

Steps performed:

- Installed Node.js and dependencies
- Cloned GitHub repository
- Started application using PM2
- Verified API endpoints using curl/browser

Application test Script:

```
#!/bin/bash
```

```
set -e
```

```
cd /home/ubuntu
```

```
# Update system
```

```
apt-get update -y
```

```
# Install required packages
```

```
apt-get install -y nginx git nodejs npm
```

```
# Clone project
```

```
git clone https://github.com/PaviTechDeveloper/Fitness_Tracker
```

```
# Configure Nginx reverse proxy
```

```
cat <<EOF > /etc/nginx/sites-available/custom
```

```
server {
```

```
    listen 80;
```

```
    server_name _;
```

```
    location / {
```

```
        proxy_pass http://localhost:5000;
```

```
        proxy_http_version 1.1;
```

```
        proxy_set_header Upgrade \${http_upgrade};
```

```
        proxy_set_header Connection 'upgrade';
```

```
        proxy_set_header Host \${host};
```

```
        proxy_cache_bypass \${http_upgrade};
```

```
    }
```

```
}
```

EOF

Enable site

ln -sf /etc/nginx/sites-available/custom /etc/nginx/sites-enabled/custom

Remove default nginx config

rm -f /etc/nginx/sites-enabled/default

Restart nginx

systemctl restart nginx

Move to backend folder

cd /home/ubuntu/Fitness_Tracker/server

Install dependencies

npm install

Install PM2

npm install -g pm2

Start application

pm2 start app.js --name "fitness-app"

pm2 save

pm2 startup systemd -u ubuntu --hp /home/ubuntu

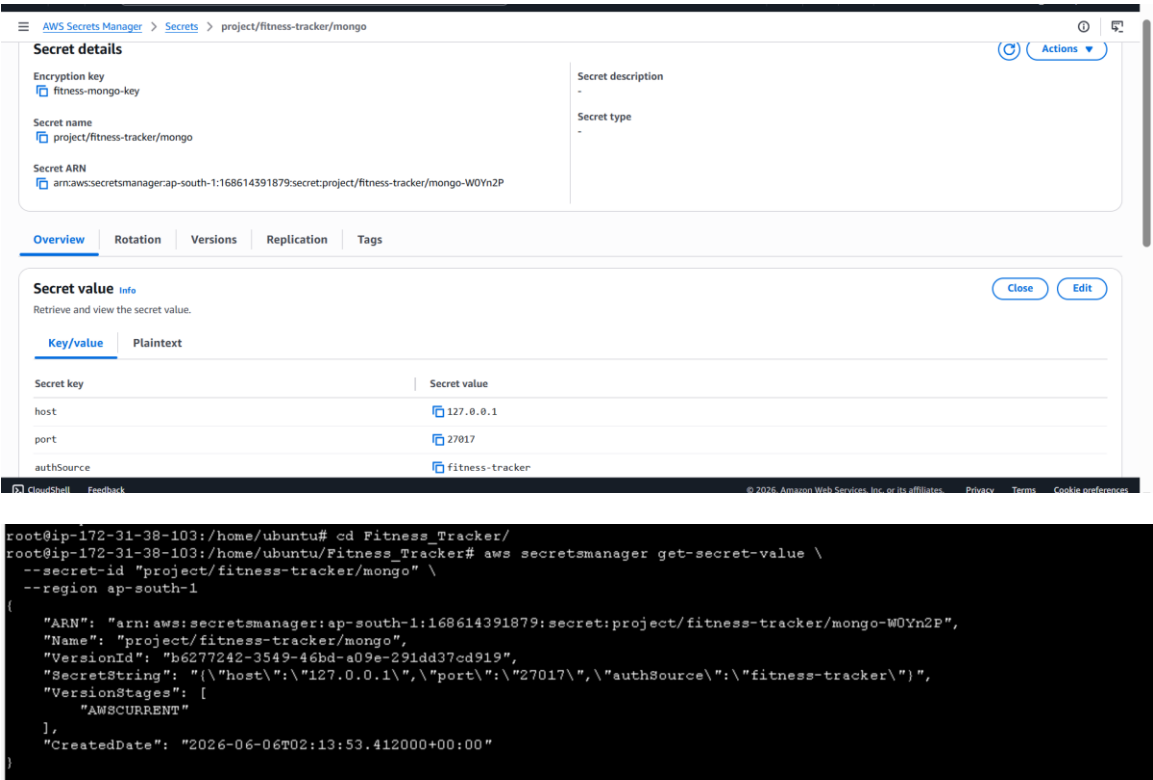
3. MongoDB Setup

MongoDB is installed on the EC2 instance and used as the primary database for storing user and fitness data.

4. AWS Secrets Manager Integration

AWS Secrets Manager is used to securely store MongoDB configuration details such as host, port, and authentication source.

Placeholder for Secrets Manager Output Screenshot:



5. AWS KMS (Key Management Service)

AWS KMS is used to encrypt secrets stored in AWS Secrets Manager, providing an additional security layer.

Placeholder for KMS Output Screenshot:

```
root@ip-172-31-38-103:/home/ubuntu/Fitness_Tracker# aws secretsmanager describe-secret \
--secret-id project/fitness-tracker/mongo \
--region ap-south-1
{
  "ARN": "arn:aws:secretsmanager:ap-south-1:168614391879:secret:project/fitness-tracker/mongo-W0Yn2P",
  "Name": "project/fitness-tracker/mongo",
  "KmsKeyId": "arn:aws:kms:ap-south-1:168614391879:key/3bad3fe2-6e44-43b8-9252-dff04096834f",
  "LastChangedDate": "2026-06-06T02:13:53.417000+00:00",
  "LastAccessedDate": "2026-06-06T00:00:00+00:00",
  "Tags": [],
  "VersionIdsToStages": {
    "b6277242-3549-46bd-a09e-291dd37cd919": [
      "AMSCURRENT"
    ],
    "e9564c80-5371-4e6d-b130-5cc51fc8b383": [
      "AMSPREVIOUS"
    ]
  }
}
```

i-001122868ad310109 (instance)

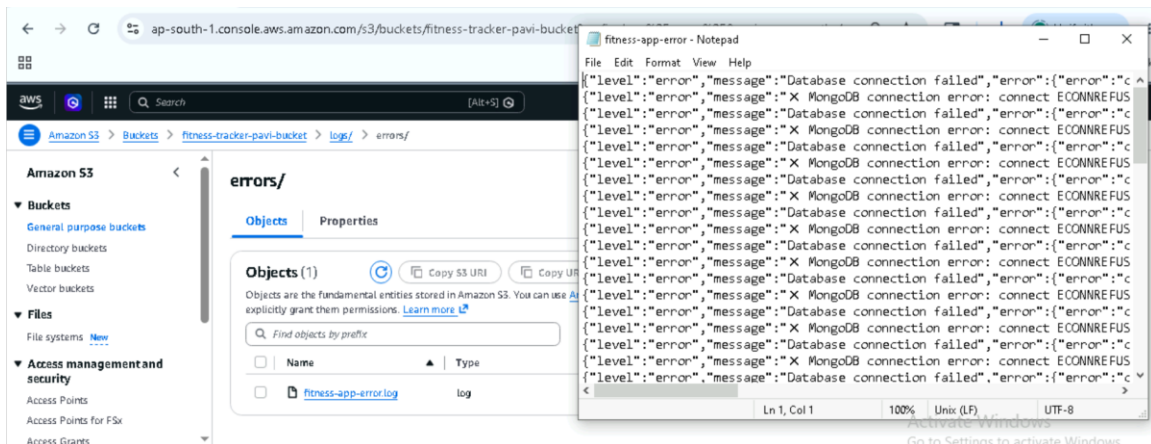
6. Amazon S3 Integration

Amazon S3 is used for storing application logs, error logs, and MongoDB backups.

S3 folder structure:

- logs/app/
- logs/errors/
- audit/auth/
- backups/mongodb/

Placeholder for S3 Output Screenshot:



7. PM2 Process Management

PM2 is used to run the Node.js application in production mode and manage logs and process lifecycle.

8. Security Architecture

The system follows a secure architecture using IAM roles, AWS Secrets Manager, and KMS encryption. No credentials are hardcoded in the application.

9. Overall Architecture Flow

User → EC2 (Node.js App) → MongoDB

↓

AWS Secrets Manager (Encrypted by KMS)

↓

Amazon S3 (Logs & Backups)